Recursão e Ordenação

Algoritmos e Estruturas de Dados

Prof. Dr. Carlos Melo

carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO
CAMPUS GARANHUNS

Recursão

Recursão

- A recursão é uma técnica que nos permite resolver um problema resolvendo instâncias menores desse problema

- Na prática ela consiste na chamada de uma função por ela mesma **(?)**:
  a. nós alimentamos uma função com um conjunto de valores;
  b. a função produz uma saída;
  c. a saída é usada como entrada pela mesma função;
  d. continuamos assim até chegarmos a solução do problema.

carlos.melo@upe.br

Leis da Recursão

1. Função recursiva deve chamar a si mesma em algum lugar (**caso recursivo**)
2. Funções recursivas têm um **caso base** ou condição de parada
   -> (tipo um laço while, mas funções recursivas não têm laços)
   -> se a função não para entramos em uma recursão infinita
3. A função recursiva deve modificar o valor de entrada em algum lugar

```
1 def recursiva(entrada):
2     if entrada <= 0:
3         return entrada
4     else:
5         saida = recursiva(entrada - 1)
6         return saida
```

2 1 3

Recursão

O esquema à esquerda é chamado de Pilha de Chamadas ou Call Stack - Lembre-se que uma pilha é do tipo LIFO

```python
def recursiva(entrada):
    if entrada <= 0:
        return entrada
    else:
        saida = recursiva(entrada - 1)
        return saida
```

~~recursiva(2)~~
~~recursiva(1)~~
~~recursiva(0)~~
return 0
saida = 0
return 0
saida = 0
return 0

Exercício

- O que acontece se eu passar como entrada o valor 10 para a função a seguir?
- E se eu passar o valor 0?
- E se eu passar um valor negativo?

```python
def recursiva(entrada):
    if entrada == 0:
        return entrada
    else:
        saida = recursiva(entrada - 1)
        return saida
```

Exercício

- Crie uma função recursiva que calcule 2^n
- Crie uma função recursiva que some todos os números inteiros positivos entre 1 e **n**
- Crie uma função recursiva que some todos os elementos de um array
- Ao invés de percorrer o array inteiro tente usar os índices para a função acima. Há alguma melhora no desempenho?

Fatorial Recursivo

- Faça a versão iterativa do fatorial que tem sua implementação recursiva a seguir:

```
1 def factorial(n):
2     if n == 0:
3         return 1
4     return n * factorial(n-1)
```

Algoritmos de Ordenação

- Os algoritmos de ordenação pertencem a uma categoria de algoritmos que colocam elementos em uma sequência ou ***ordem***
- Esta ordem pode ser numérica, lexicográfica ou por qualquer outra característica pré-definida
- As principais razões para se ordenar elementos são:
  - facilitar visualização ou
  - otimizar buscas.

Algoritmos de Ordenação

- Você recebe uma coleção de músicas ou uma lista de contatos, se não houvesse ordem, quão trabalhoso seria encontrarmos algo que estamos procurando como usuários?
- Existem vários algoritmos de ordenação, alguns mais rápidos porém complexos de se implementar, já outros mais fáceis de implementar, porém muito mais lentos.
- Vamos conhecer alguns!

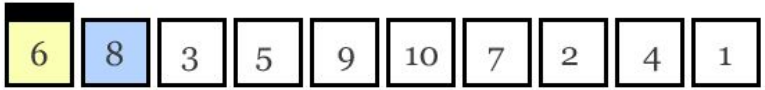
Algoritmos de Ordenação de Força Bruta

- Força Bruta (abordagem ingênua) é uma técnica que testa cada possibilidade existente, uma a uma, até resolver um problema
  - Lembra do nosso cadeado da primeira aula? Se ele tiver 4 dígitos de 0-9 podemos precisar testar 10^4 ou 10000 combinações (0000-9999) até encontrar o segredo que o abre (isso é um exemplo de força bruta)
- Quanto aos algoritmos de ordenação que usam a força bruta os principais são:
  - Selection Sort,
  - Insertion Sort e
  - Bubble Sort

Selection Sort (Ordenação por Seleção)

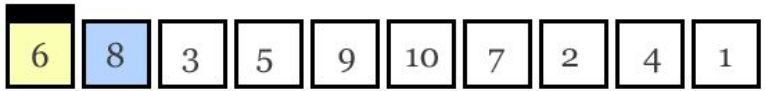
- A ideia do Selection Sort é bem intuitiva:

1. Para cada posição da lista:
2. Encontrar o menor elemento da parte não ordenada
3. Trocar esse elemento com o elemento da posição atual
4. Repetir até o final da lista



Yellow is smallest number found
Blue is current item
Green is sorted list

Exercício: Implemente o Selection Sort

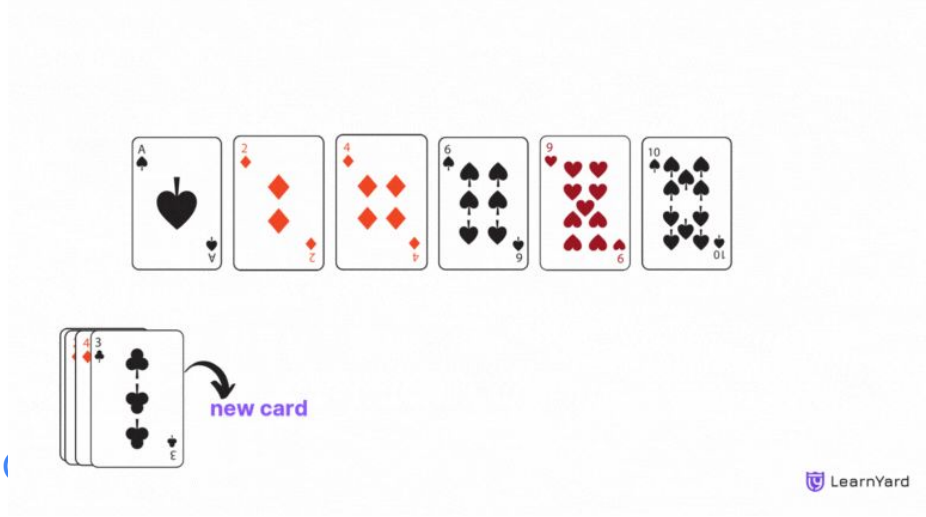


Yellow is smallest number found
Blue is current item
Green is sorted list

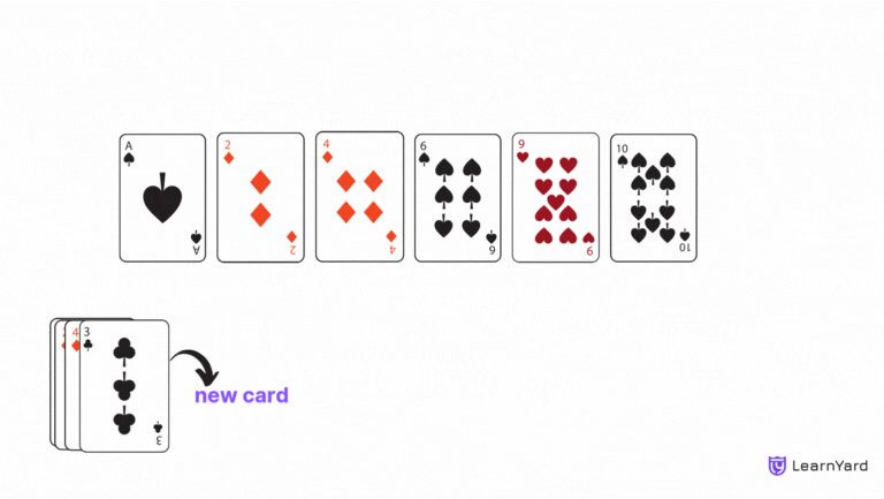
Insertion Sort (Ordenação por Inserção)

- A ordenação por inserção é mais simples que a ordenação por seleção
- A ideia é:

1. Considerar que o primeiro elemento já está ordenado
2. Pegar o próximo elemento (chave)
3. Comparar com os elementos anteriores
4. Deslocar os maiores para a direita
5. Inserir a chave na posição correta
6. Repetir o processo até o final da lista



Exercício: Implemente o Insertion Sort



carlos.melo@upe.br

Bubble Sort

1. Comece pelo início e compare os dois primeiros elementos
2. Cheque se o próximo elemento é menor que o elemento atual
   a. Se sim, troque os elemento na posição atual (flutuação/bubble)
   b. Se não, não troque e continue
3. Repita o processo até que todos os elementos estejam ordenados

Exercício: Implemente o Bubble Sort

Prof. Dr. Carlos Melo
https://calendly.com/prof-casm
carlos.melo@upe.br

UPE UNIVERSIDADE DE PERNAMBUCO CAMPUS GARANHUNS